

TEST AUTOMATION PROJECT REPORT

Ixigo Flight Booking Automation - BDD

Behavior Driven Development with Behave, Gherkin, Selenium WebDriver, POM, Allure and custom HTML reporting

Application under test: ixigo flight booking web journey

Test design approach: Behavior Driven Development using Gherkin scenarios

BDD framework: Behave feature files with Selenium-backed step definitions

Automation pattern: Page Object Model for browser/page actions

Execution browser: Microsoft Edge or Chrome, selected by environment hook

Report date: 23 May 2026

Behaviour coverage: 24 BDD scenarios across combined flow and full end-to-end booking flow

1. Synopsis

This project converts the ixigo Selenium automation into a BDD-style Behave framework. The tested behaviour remains the same: search flights, validate results, apply filter and sort actions, handle invalid search input, move into booking, enter traveller details, pass through add-ons and reach the payment page. The difference is in how the test intent is expressed. Instead of only Python test methods, the BDD version presents the flow through readable Gherkin scenarios and keeps the Selenium work inside step definitions.

The structure is deliberately close to the Selenium project so the evaluator can see that the framework changed while the business journey stayed consistent. The BDD code has two feature files and two step-definition folders: one for the combined flow and one for the richer full end-to-end booking flow.

2. Project Scope

- Represent the major ixigo user behaviours in Gherkin using Given, When and Then steps.
- Keep assertions inside step definitions so each Then step validates a real browser state.
- Reuse the Page Object Model for browser interaction rather than placing Selenium locators in feature files.
- Run the combined search, negative and booking smoke checks separately from the full E2E booking flow.
- Capture screenshots during important actions and after each scenario for evidence.

- Generate Allure result files and a custom local HTML report after the Behave run.

3. BDD Architecture

The code uses BDD and POM together. BDD describes the expected behaviour in feature files; POM organises the Selenium implementation behind those steps. This means the evaluator can read the scenario intent without seeing locators, while the automation code remains maintainable.

Layer	Actual code location	Responsibility
Feature files	features/combined_flow.feature, features/e2e_full_booking.feature	Define behaviours in readable Gherkin language.
Combined steps	features/steps/combined_flow_steps/combined_steps.py	Search, result validation, filter, sort, negative scenarios and booking smoke checks.
E2E steps	features/steps/e2e_full_booking_steps/e2e_steps.py	Login, traveller details, add-ons, seat selection and payment checks.
Hooks	features/environment.py	Browser lifecycle, report cleanup, screenshots, HTML report generation and Allure opening.
Page objects	pages/	Encapsulate Selenium interactions for search, results, booking, add-ons, payment and login.
Data/utilities	data/, utils/	CSV test data, route reading, config reading, logging and screenshot capture.

4. Behaviour Files

The BDD project has two feature files. The combined feature checks the smaller behaviours that would normally be reviewed as separate regression checks. The full end-to-end feature keeps the booking journey in order so it looks like the Selenium E2E flow, but described as business behaviour.

Feature	Scenario count	Main behaviours
Ixigo combined flow	9	Open page, verify controls, search BOM to BLR, verify book buttons, filter/sort, print results, invalid source/destination, missing destination and booking-page smoke flow.
Ixigo full end-to-end booking	15	Open site, login, search random CSV route, filter, sort, print results, book, booking page, cancellation prompt, traveller details, add-ons, seat selection, skip to payment and payment page.

5. Assertion Strategy

The feature files are intentionally readable and do not contain Python assertions. The assertions are implemented in the step-definition files, which is the normal Behave pattern. The Gherkin says what should happen; the Python step code proves it using Selenium and page objects.

BDD check	Where it is asserted	Validation performed
Page loaded	combined_steps.py	Browser title contains ixigo.
Search controls visible	combined_steps.py	From, To and Search elements are present.
Matching results displayed	combined_steps.py	Results load and flight count is greater than zero.
Visible flights remain	combined_steps.py	Flight cards and Book buttons are visible.
Prices sorted	combined_steps.py	First visible prices are in ascending order when prices are parsed.
Booking page displayed	combined_steps.py	Booking URL or traveller/contact content is detected.
Traveller form filled	e2e_steps.py	CSV traveller details are entered and form fill returns true.
Payment page displayed	e2e_steps.py	Payment page wait condition returns true.
Negative route checks	combined_steps.py	Invalid route does not navigate to accepted result URL.

6. Execution Design

The hook file starts one browser per feature, matching the Selenium-style ordered flow. Scenarios tagged as isolated can still use their own browser, which keeps negative checks from damaging the main journey. Before every run, report folders and logs are cleaned so the output represents the latest execution.

- Nested step modules are enabled in behave.ini, so Behave loads both step-definition folders directly.
- The default formatter writes Allure result files to reports/allure-results.
- environment.py patches Allure behaviour labels, builds a custom HTML report and opens it after execution.
- If the Allure CLI is unavailable, the local HTML report still opens and the test run does not fail because of reporting.
- Screenshots are captured both during key actions and at scenario completion.

7. Test Environment

Area	Details
Language	Python
Test design approach	Behavior Driven Development with Given, When and Then scenarios
BDD framework / runner	Behave
Browser automation tool	Selenium WebDriver
Automation design pattern	Page Object Model
Step discovery	Nested step modules enabled in behave.ini
Reports	Allure result files, generated Allure report when CLI is installed, and custom report.html

Browser	Configured browser with Edge/Chrome fallback in environment.py
Data	CSV files for route, login and traveller input

8. BDD Scenario Summary

No.	Feature	Scenario	BDD validation	Status
1	Combined	Open ixigo page	Then page title confirms ixigo loaded	Passed
2	Combined	Page elements are present	Then From, To and Search controls are visible	Passed
3	Combined	Search BOM to BLR flights	Then matching results and visible flights remain	Passed
4	Combined	Flight count and book buttons	Then flights and at least two Book buttons are visible	Passed
5	Combined	Apply nonstop filter and sort by price	Then flights remain and parsed prices are sorted	Passed
6	Combined	Print flight results	Then at least one result remains	Passed
7	Combined	Same source and destination	Then bom-bom route is not accepted	Passed
8	Combined	Search without destination	Then user remains off results page	Passed
9	Combined	Booking flow reaches traveller details	Then booking page is displayed after booking action	Passed
10	E2E	Open site	Then ixigo page should be loaded	Passed
11	E2E	Login	When mobile row is submitted and OTP wait completes	Passed
12	E2E	Search flights	Then matching flight results are displayed	Passed
13	E2E	Apply nonstop filter	Then at least one visible flight remains	Passed
14	E2E	Sort by price	Then at least one visible flight remains	Passed
15	E2E	Print results	Then at least one visible flight remains	Passed
16	E2E	Click book	When first visible flight is booked	Passed
17	E2E	Booking page	Then booking page should be displayed	Passed
18	E2E	Decline cancellation	When cancellation prompt is declined	Passed
19	E2E	Fill traveller	Then traveller form fill assertion passes	Passed
20	E2E	Continue and confirm	When details are continued and confirmed	Passed
21	E2E	Add-ons page	Then add-ons page should be displayed	Passed
22	E2E	Select seat	When a seat is selected if available	Passed
23	E2E	Skip to payment	When add-ons are skipped to payment	Passed
24	E2E	Payment page	Then payment page should be displayed	Passed

9. Screenshot Evidence

The screenshots below are selected from the latest BDD run artifacts under logs. They are arranged in journey order so the report reads like the actual behaviour flow rather than a random screenshot dump.

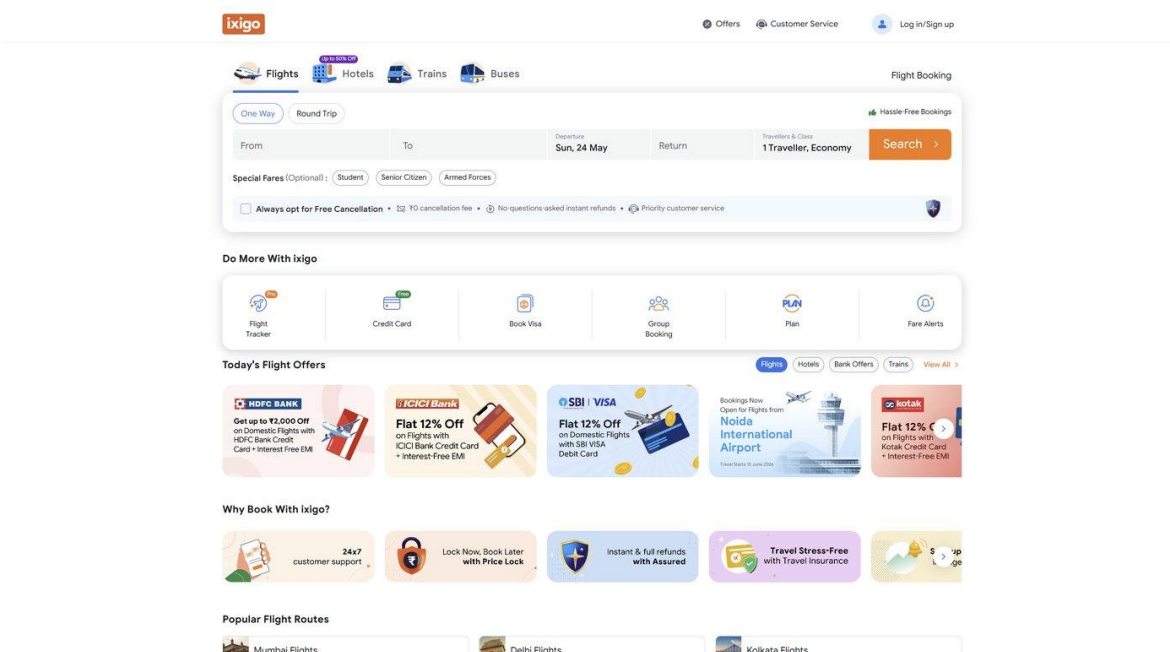


Figure 1. BDD scenario opens the ixigo flight page.

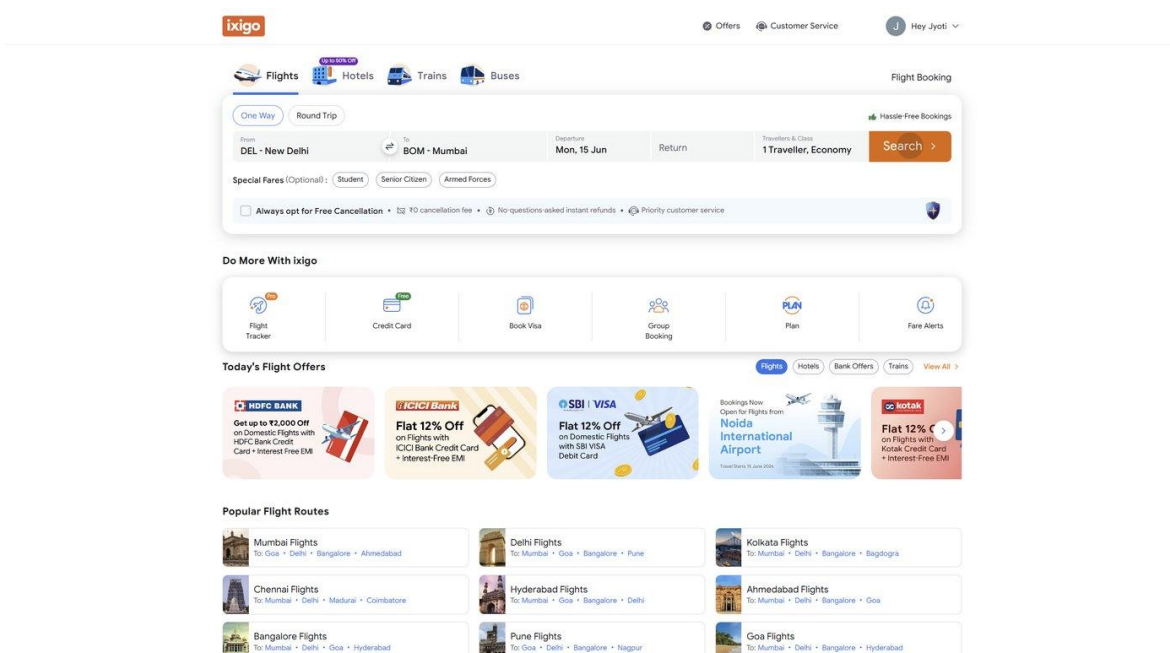


Figure 2. Route search executed from a BDD step.

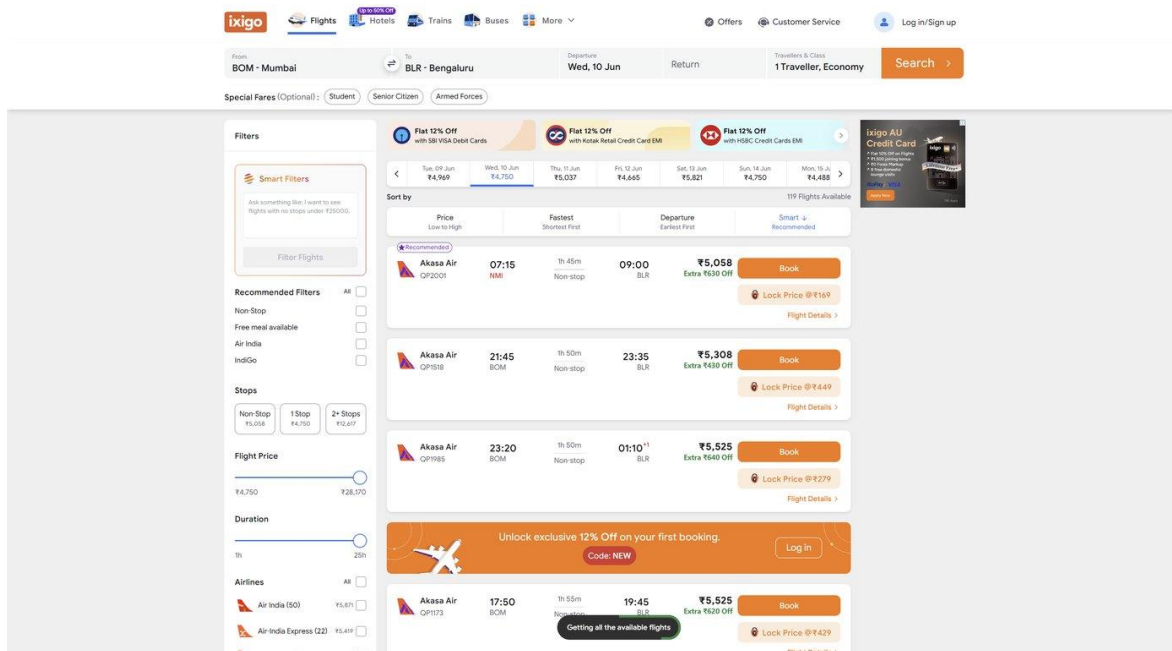


Figure 3. Combined flow validates that flight results are displayed.

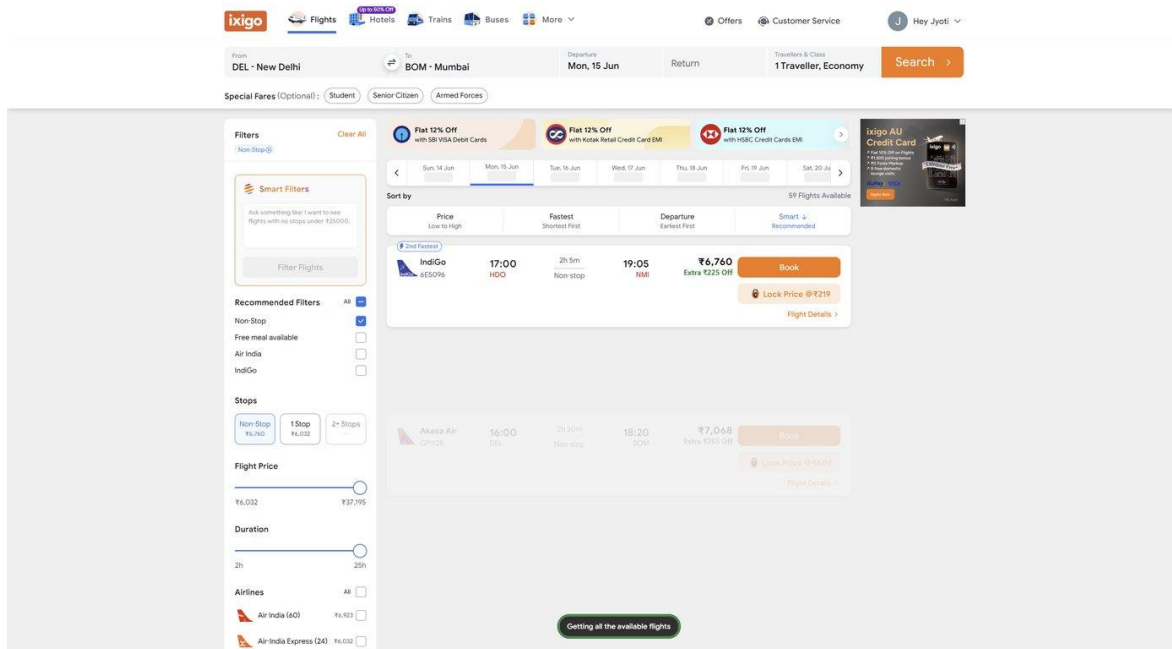


Figure 4. Non-stop filter applied during the behaviour flow.

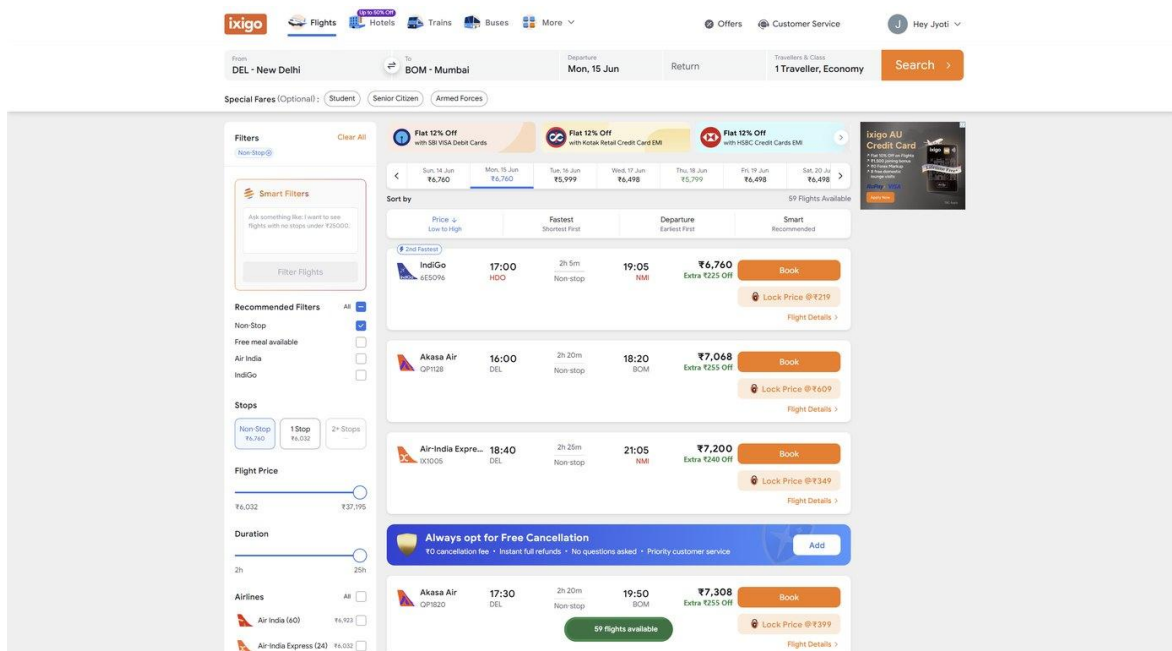


Figure 5. Price sorting evidence after filtering.

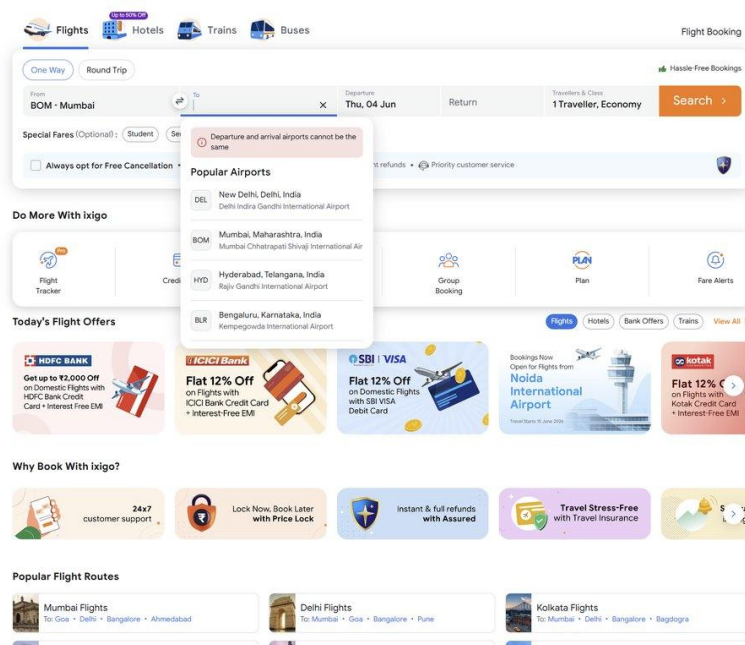


Figure 6. Negative behaviour confirms same source and destination are not accepted.

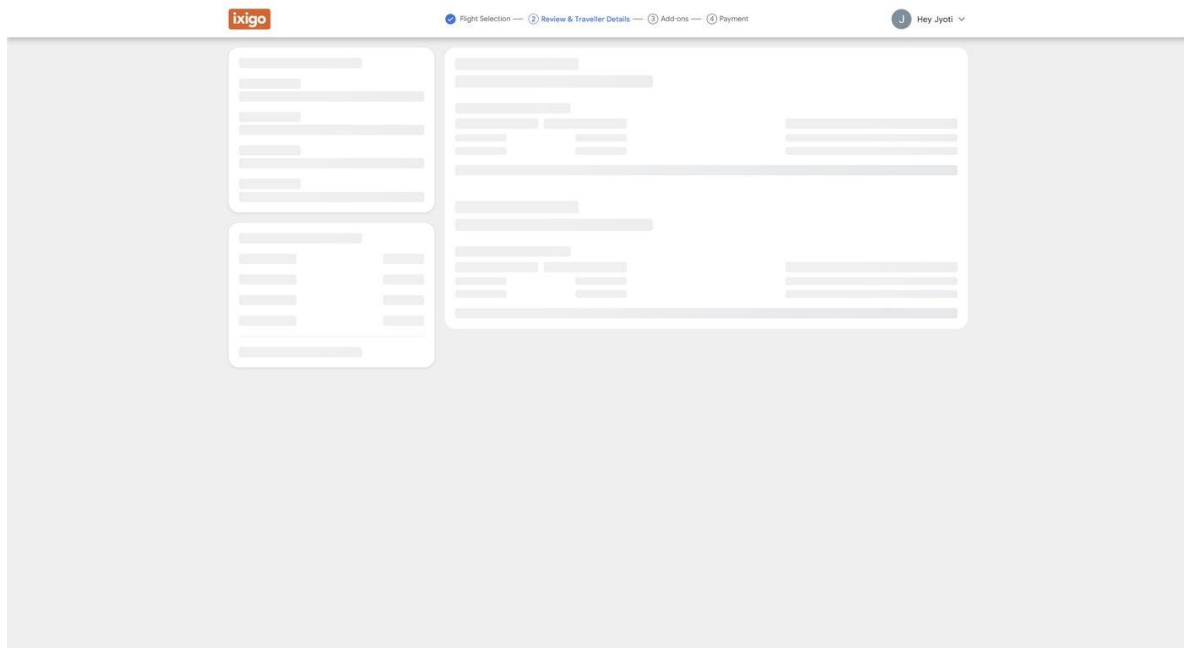


Figure 7. Booking action triggered from the result list.

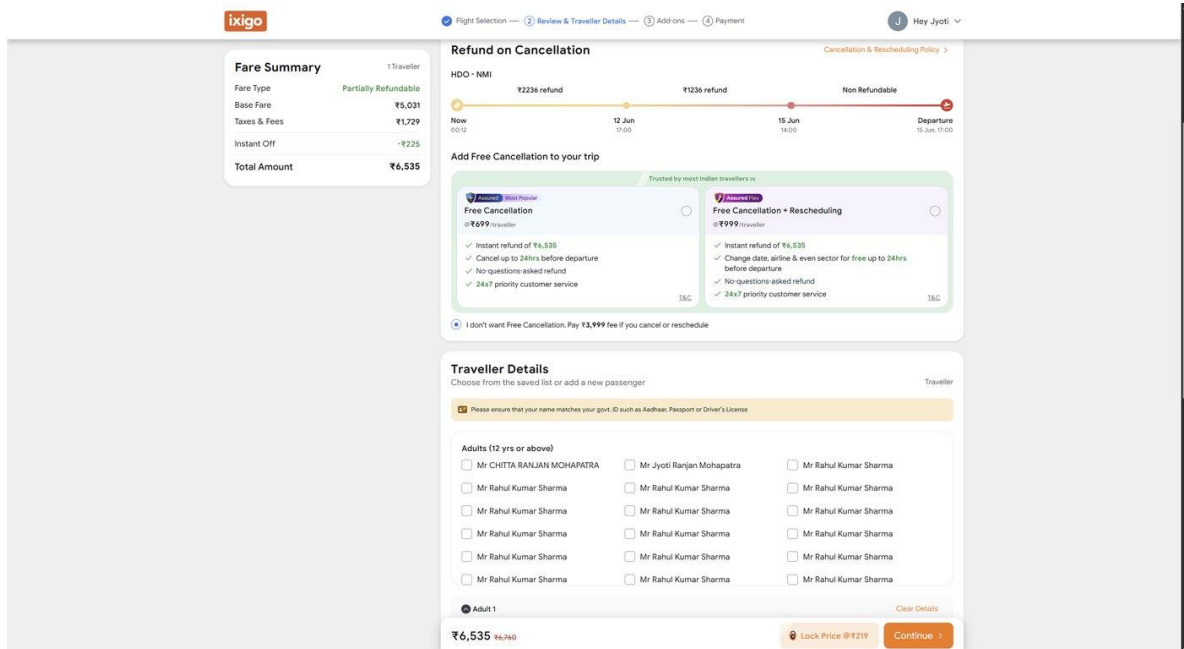


Figure 8. Cancellation prompt handled in the booking journey.

ixigo Flight Selection — **Review & Traveller Details** — Add-ons — Payment Hey Jyoti

Fare Summary 1 Traveller
Fare Type: Partially Refundable
Base Fare: ₹5,031
Taxes & Fees: ₹1,729
Instant Off: -₹225
Total Amount: ₹6,535

Contact Details
Your ticket & flight information will be sent here
Country Code: India (+91) Mobile Number: 9876543210 Email: rahul.sharma@example.com

GST Details
To claim credit of GST charged by airlines, please enter your GST details
☐ I would like to add my GST Number

Billing Address
As per the latest govt. regulations, it's mandatory to provide your address.
Pincode: 560001 Address: 42 MG Road Bengaluru
City: Bengaluru State: Karnataka

By clicking on continue, I confirm that I have read, understood, and agree with the [Fare Rules](#), [Privacy Policy](#) and [Terms of Use](#).
100% Safe Payment Process
Verified by Visa SafeKey MasterCard SecureCode

₹6,535 ~~₹6,760~~ Lock Price @₹219 Continue

Figure 9. Traveller details filled from CSV data.

ixigo Flight Selection — **Review & Traveller Details** — **Add-ons** — Payment Hey Jyoti

Your Flight One Way
Mon, 15 Jun
17:00 Ghaziabad 2h 5m Non stop 19:05 New Mumbai

Travellers
1 Mr. Rahul Kumar Sharma

Fare Summary 1 Traveller
Fare Type: Partially Refundable
Base Fare: ₹5,031
Taxes & Fees: ₹1,729
Instant Off: -₹225
Total Amount: ₹6,127

Seat Map
Seat Meal Insurance
Price: ₹425 - ₹475 ₹725 - ₹945
Front view of the aircraft showing seats 1-7 in rows B-F. A callout for 'Rahul Kumar S' is visible.

₹6,127 ~~₹6,360~~ Meal Selection

Figure 10. Add-ons page reached after confirming traveller details.

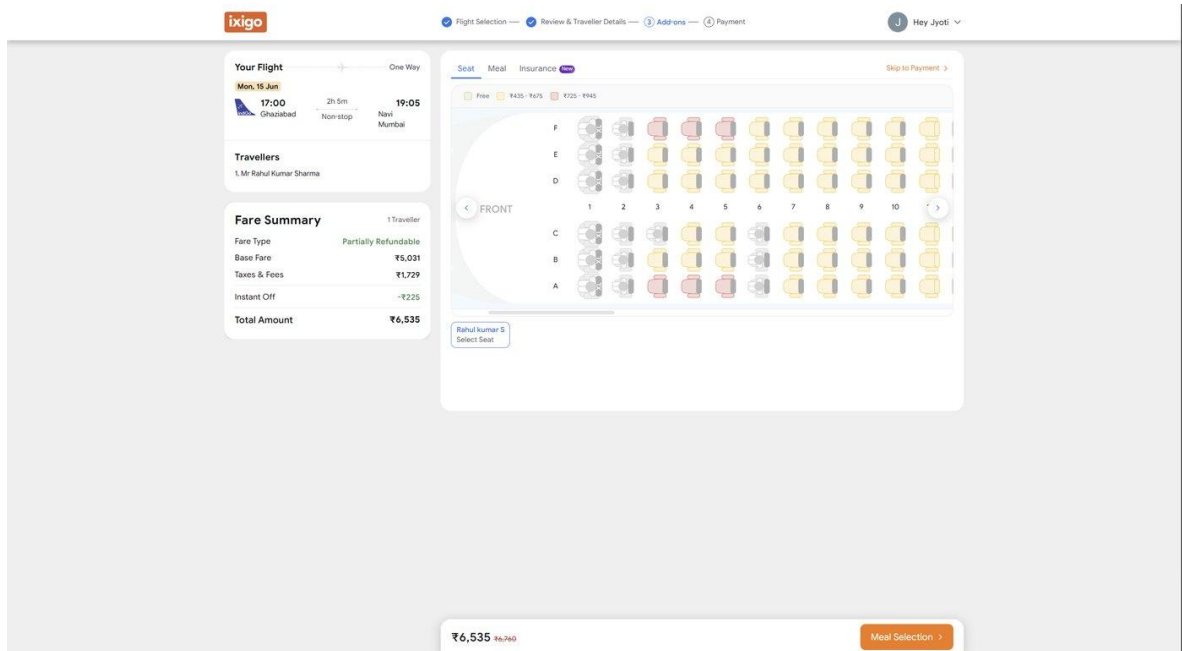


Figure 11. Seat selection behaviour executed where available.

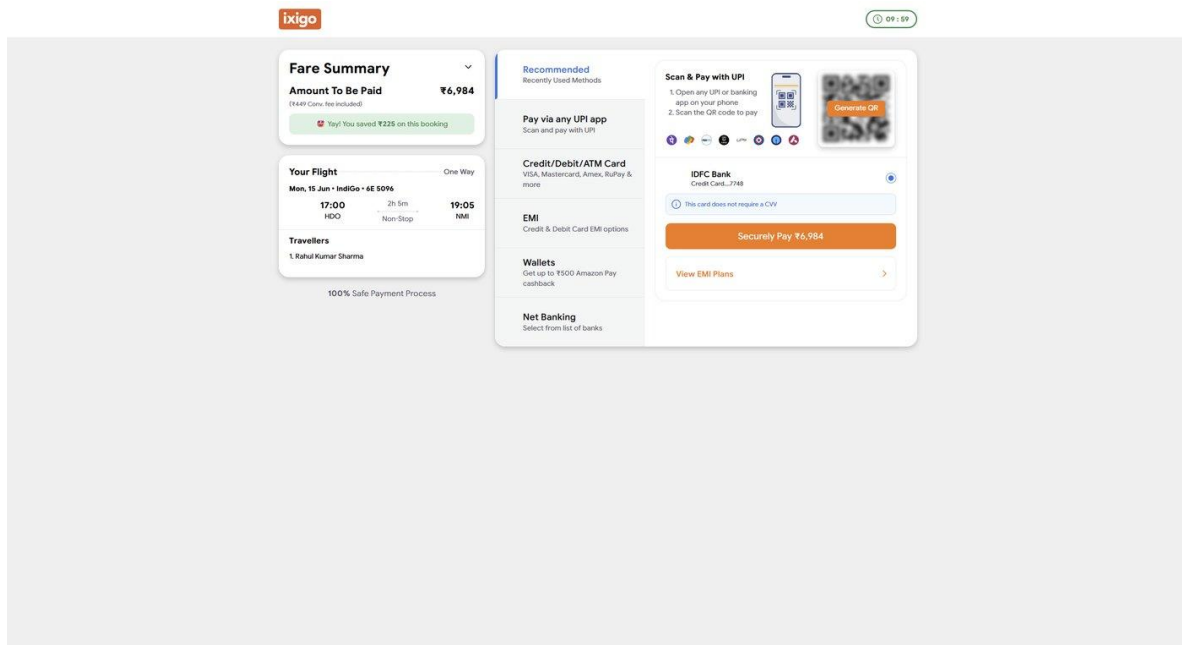


Figure 12. Payment page reached at the end of the BDD E2E flow.

10. Observations

- BDD is documented as the test design approach, while POM is documented as the automation code design pattern.
- The BDD layer makes the testing intent easier to explain because feature files read like user behaviour.

- The actual validations are still strong because Selenium assertions live inside the step-definition code.
- The two step folders give a clean evaluation structure: combined behaviours separately from the E2E booking journey.
- The browser lifecycle follows the Selenium style, with one feature-level browser and isolated browsers only where needed.
- The reporting hook keeps the project portable by opening HTML even when Allure CLI is missing.

11. Conclusion

The BDD project preserves the Selenium automation coverage while presenting it in a behaviour-driven format. The feature files communicate the business flow, the step folders keep implementation organised, and the assertions remain in Python where Selenium can verify the page state properly. The result is a Behave-based framework that is suitable for a capstone submission because it shows both testing discipline and readable behaviour coverage.